

CSE 4403: Algorithms

Lecture16

Divide and Conquer - 1

Anika Farzana

Junior Lecturer

Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

Email: anikafarzana@iut-dhaka.edu

Divide and Conquer

Break a big problem into smaller problems, solve those, combine the results.

So what is new today?

In Dynamic Programming, subproblems overlapped — the same smaller problem appeared repeatedly, so we cached results to avoid recomputing.

Today, our subproblems are independent. No overlap. No caching needed.

DP vs Divide & Conquer — The Key Difference

	Dynamic Programming	Divide & Conquer
Subproblems	Overlap	Independent
Same problem twice?	Yes — cache it	No — solve once
Direction	Bottom-up table	Top-down recursion
Examples	Coin change, Knapsack, LCS	Merge Sort, Binary Search

In DP, we cached because the same subproblem appeared repeatedly. In D&C, subproblems never repeat — nothing to cache.

The General Template

```
solve(problem) :  
    if problem is small enough:  
        return base case solution      ← base  
case  
  
    left, right = split(problem)  
  
    left_ans  = solve(left)             ← recurse  
    right_ans = solve(right)           ← recurse  
  
    return combine(left_ans, right_ans) ← combine
```

Split

Divide into independent subproblems

Solve

Recurse on each part

Combine

Merge results into final answer

*Every D&C algorithm is just a different answer to:
"How do you split?" and "How do you combine?"*

The Hard Part Is Never the Split

In most D&C algorithms:

The SPLIT is dumb

Just cut in half.
No intelligence required.

The COMBINE is smart

This is where the real work lives.
Most of the algorithm's logic is here.

Examples:

Merge Sort

Split: Cut array in half — trivial

Combine: Merge two sorted halves — $O(n)$ work

Binary Search

Split: Find midpoint — trivial

Combine: No combine. Just pick one half

Merge Sort

```
void mergeSort(vector<int>& arr, int l, int r) {  
    if (l >= r) return;           // base case  
  
    int mid = (l + r) / 2;  
  
    mergeSort(arr, l, mid);       // sort left half  
    mergeSort(arr, mid+1, r);     // sort right half  
  
    merge(arr, l, mid, r);       // combine  
  
}
```

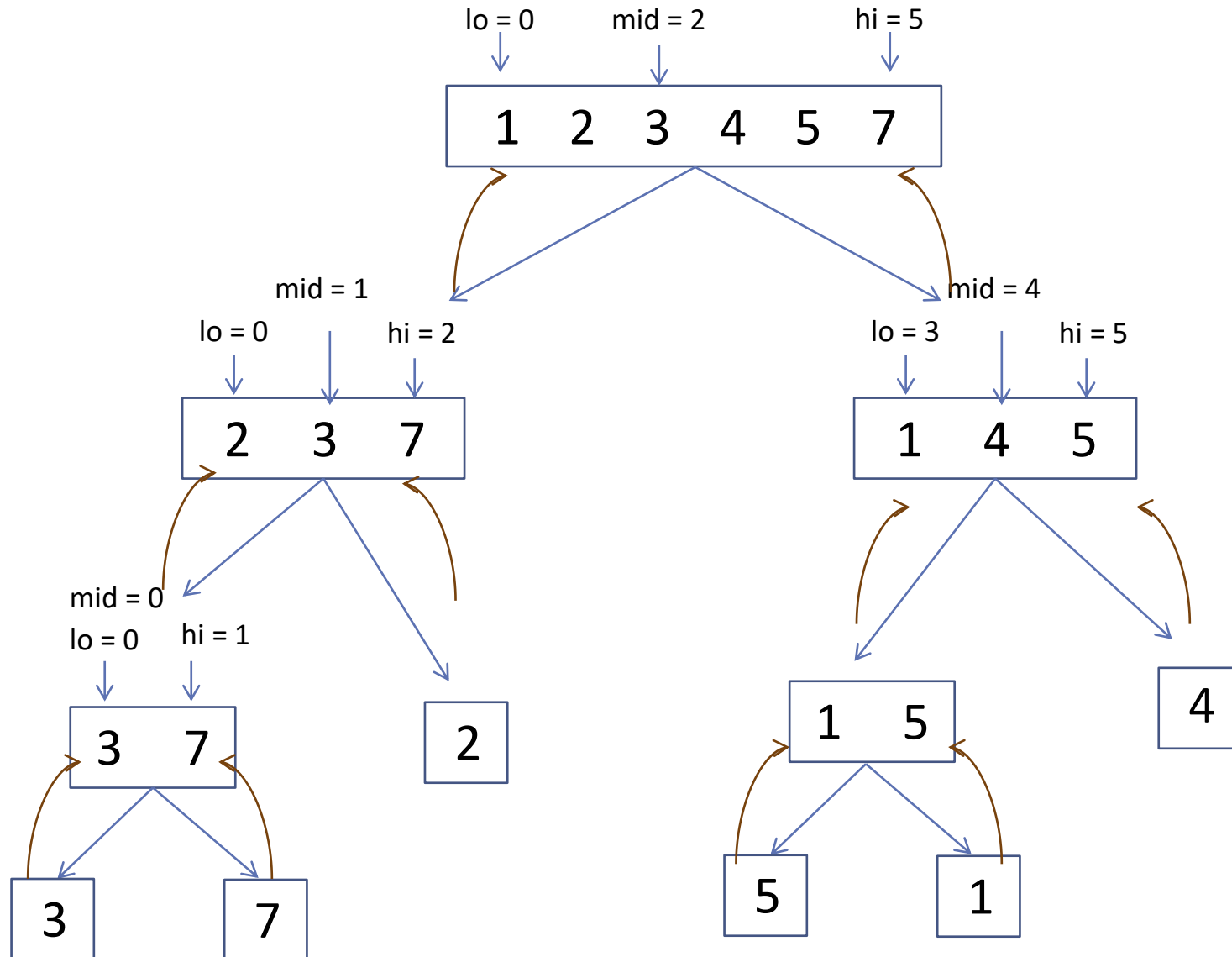
← Base case

← Split and Recurse

← Split and Recurse

← Combine step

Merge Sort: Recap



Why $O(n \log n)$?

Visual Explanation

The recursion tree has $\log n$ levels
(we halve each time).

At every level, the total work done
across all merges = $O(n)$.

So total = $O(n) \times \log n = O(n \log n)$.

Recurrence Relation

$$T(n) = 2T(n/2) + O(n)$$

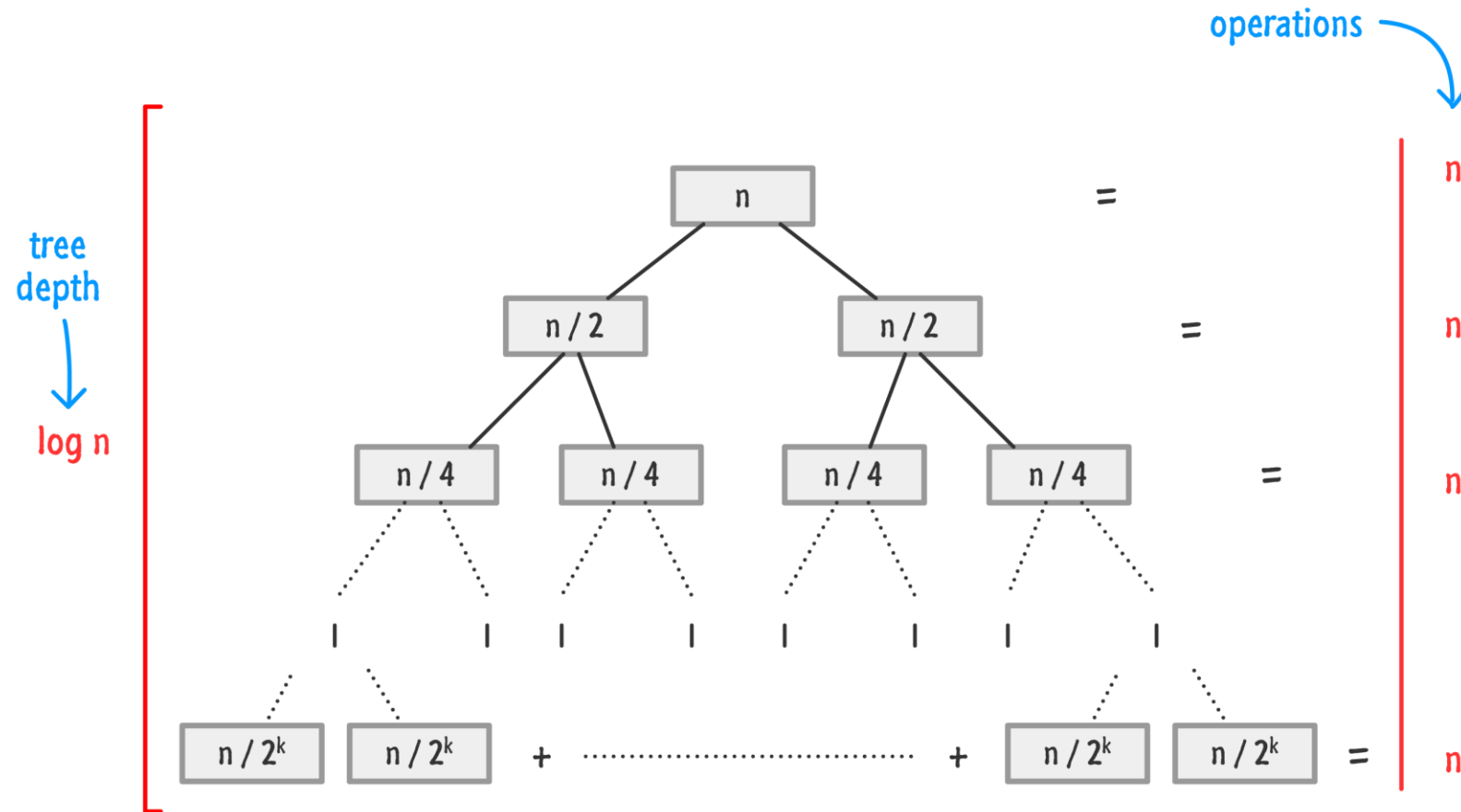
$2T(n/2) \rightarrow$ two recursive calls on half-sized arrays

$O(n) \rightarrow$ the merge step

Solves to $O(n \log n)$.

(We will Prove formally with Master Theorem)

Merge Sort Tree



Home Task

Binary Search — Reframe it as D&C

```
solve(arr, l, r, target):  
    if l > r: return -1                // base case: not found  
  
    mid = (l + r) / 2  
  
    if arr[mid] == target:  
        return mid                    // found!  
    else if arr[mid] > target:  
        return solve(arr, l, mid-1, target)    // left half  
    else:  
        return solve(arr, mid+1, r, target)    // right half
```

Think about:

1. What is the "combine" step here? (Hint: is there one?)
2. What is the recurrence relation and time complexity?
3. What does the recursion tree look like?

Generalized

Given a problem of size n

Divide into a subproblems

Each with size $\frac{n}{b}$

Where $a \geq 1$ and $b > 1$

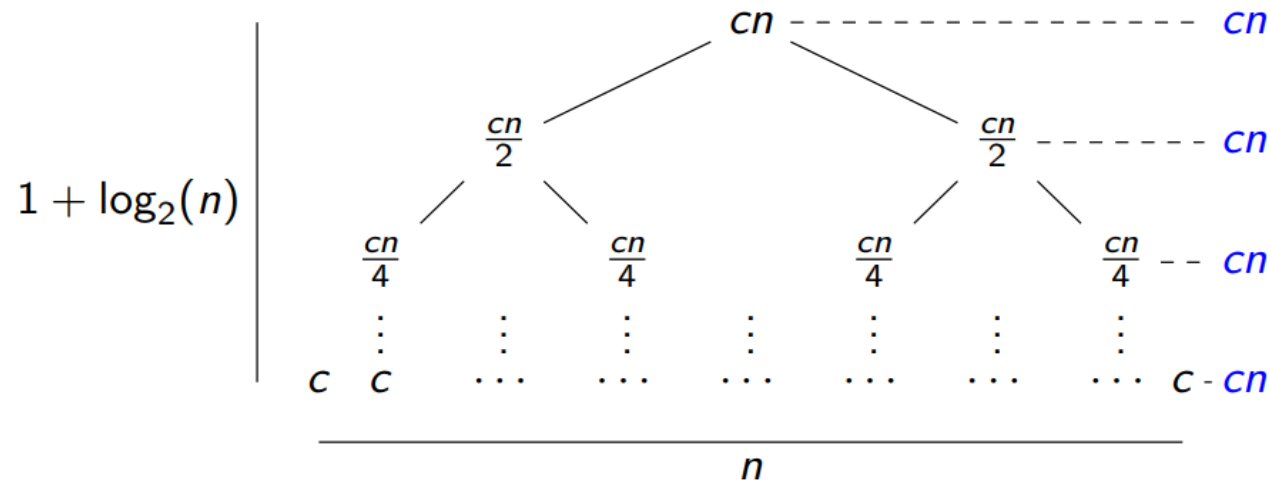
Solve each subproblem recursively

Combine the solutions

Running time: $T(n) = aT\left(\frac{n}{b}\right) + [merge]$

Example - 1

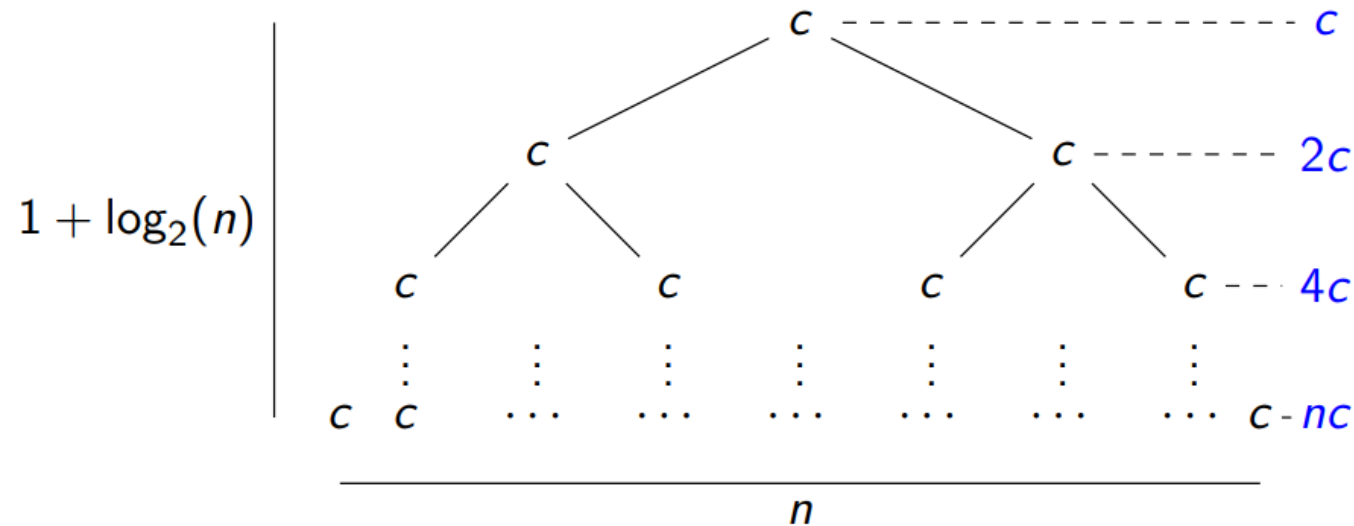
- Solve $T(n) = 2T\left(\frac{n}{2}\right) + cn$, where $c > 0$ is constant
- Recursion Tree



- Equal amount of work done in each level
- Total: $cn \times (1 + \log_2(n)) = cn + cn \log_2(n) = \Theta(n \log_2(n))$

Example - 2

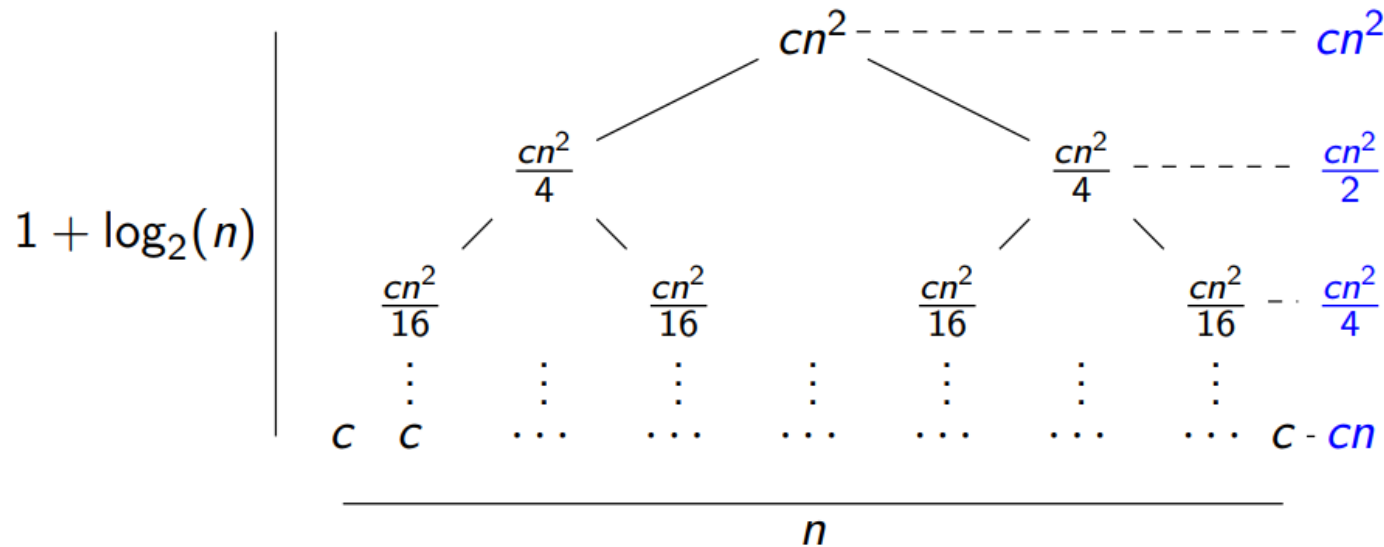
- Solve $T(n) = 2T\left(\frac{n}{2}\right) + c$, where $c > 0$ is constant



- Most of the work done in leaves
 - $T(n) = nc + \frac{nc}{2} + \frac{nc}{4} + \dots + \frac{nc}{n/2} + \frac{nc}{n}$
- $\Rightarrow T(n) = nc \left(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{n/2} + \frac{1}{n}\right)$
- $\Rightarrow T(n) = \Theta(n)$

Example - 3

- Solve $T(n) = 2T\left(\frac{n}{2}\right) + cn^2$, where $c > 0$ is constant



- Most of the work done in root
- $T(n) = cn^2 + \frac{cn^2}{2} + \frac{cn^2}{4} + \dots + \frac{cn^2}{n}$
- $\Rightarrow T(n) = cn^2 \left(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{n}\right)$
- $\Rightarrow T(n) = \Theta(n^2)$

Master Theorem

$$T(n) = aT(n/b) + f(n) \text{ where } a \geq 1 \text{ and } b > 1$$

There are following three cases:

1. If $f(n) = O(n^c)$ where $c < \log_b a$ then $T(n) = \Theta(n^{\log_b a})$
2. If $f(n) = \Theta(n^c)$ where $c = \log_b a$ then $T(n) = \Theta(n^c \log n)$
3. If $f(n) = \Omega(n^c)$ where $c > \log_b a$ then $T(n) = \Theta(f(n))$